Woojin Ji (random6)

Linux kernel & eBPF security | Rust / C

GitHub · Technical blog · Email

Profile

Open-source contributor focused on Linux eBPF verification and systems security. Authored four commits in mainline Linux across the BPF verifier, selftests, and bpftool; builds Rust and C tooling for differential testing, kernel experiments, and endpoint telemetry.

Open-source contributions

Linux kernel, BPF subsystem · 2026

- **BPF verifier:** Preserved known-zero stack reads across scalar zero spills while tracking spill precision for sound pruning. Authored the kernel fix and regression selftests (patch series).
- **BPF selftests:** Added an arena regression test for a direct-value pseudo load at the one-past-end offset (patch discussion).
- **bpftool:** Fixed flow-dissector query handling to use libbpf's error code rather than `errno` after `close()`, preserving old-kernel compatibility (patch discussion).

CTF

DEF CON 33 CTF Finals (2025) — Finalist with **cold fusion**; competed in pwn and reverse engineering.

Selected projects

- **rand-verifier** · Rust — Differential testing for the Linux eBPF verifier, combining a path-sensitive abstract interpreter, bounded concrete execution, an independent safety specification, and real-kernel verdicts. Includes fuzzing, test-case reduction, SMT checks, and a bpf-next QEMU test path.
- **kernel-ebpf-lab** · Buildroot, QEMU — Reproducible kernel builds and guest boots for BPF patch work, with SSH-based smoke tests and verifier-response collection.
- **rand-guard** · Rust, Aya — An eBPF endpoint detection prototype with process and file telemetry, opt-in network events, userspace enrichment, focused detections, and JSON Lines output (project write-up).

Technical focus

Languages: Rust, C
Systems and security: Linux kernel, eBPF verifier, BPF selftests, endpoint telemetry, low-level security
Tools and methods: Aya, libbpf, LLVM/Clang, QEMU, Buildroot, fuzzing, differential testing

Selected writing

- Patching the eBPF Verifier to Preserve Spilled Zeroes — How the upstream verifier fix preserves precision without breaking pruning.
- Why 12 eBPF Partial-Init Variants That Looked Equivalent Produced Different Verification Results — Controlled experiments on stack initialization and verifier behavior.

Education

Dankook University, Jukjeon Campus — B.Eng. in Cybersecurity
Expected graduation: February 2028